



@sovit.tech.knowledge

■ JAVA · SPRING BOOT · 2026 EDITION

Backend Development Roadmap 2026

A week-by-week guide to your first backend job

From core Java fundamentals to production-ready REST APIs, databases, Git and system design — a structured 12-week path with resources, milestones and projects.

🕒 12 Weeks

🌱 6 Phases

☕ Java

🌱 Spring Boot

📄 SQL

🐙 Git

📐 System Design



What's Inside

6 phases · 12 weeks · build 5 real projects along the way

This roadmap turns "I want to be a backend developer" into a concrete daily plan. Each phase builds on the last — don't skip ahead. The goal isn't to finish fast, it's to build things you understand. One focused hour every day beats six hours on a weekend.

01	Core Java Fundamentals Data types, control flow, OOP foundations & Collections	Weeks 1–2
02	Java Deep Dive & Modern Features Generics, Streams, Lambdas, Records, modern APIs	Week 3
03	Spring Boot & REST API Development DI, controllers, REST design, validation, testing	Weeks 4–6
04	Databases, SQL & JPA / Hibernate SQL, PostgreSQL, entities, relationships, migrations	Weeks 7–8
05	Git, Tooling & Real-World Projects Professional Git workflow + a full capstone project	Weeks 9–10
06	Interview Prep & System Design DSA, Java/Spring review, entry-level system design	Weeks 11–12
A	Resources Master List Every platform, channel, book and tool in one place	Appendix
B	Projects Reference What each project demonstrates + build time	Appendix
C	Interview Checklist The questions you must be able to answer	Appendix

01

Core Java Fundamentals

Weeks 1–2 · The non-negotiable foundation

Before touching any framework, be comfortable with Java as a language. Most beginners rush past this phase and then struggle with Spring Boot. Spend two full weeks here and you'll save yourself weeks of confusion later. Focus on understanding how things work, not just on making them work.

WEEK 1 Java Basics & Control Flow

TOPICS TO COVER

- Setting up **JDK 21 + IntelliJ IDEA** (Community Edition is free)
- Data types, variables, type casting — `int`, `long`, `double`, `String`, `boolean`
- Operators: arithmetic, relational, logical, bitwise
- Control flow: `if/else`, switch expressions (Java 14+ style), ternary
- Loops: `for`, `while`, `do-while`, enhanced for-each
- Arrays: single-dimensional and 2D, iterating, common operations
- Methods: defining, calling, return types, method overloading
- `Scanner` for user input — write small interactive programs

Daily practice: spend the first 30 minutes of each session writing code, not watching videos. After every concept, write 2–3 small programs from scratch without looking. The goal is muscle memory — you should write a basic loop or method without thinking.

RESOURCES

PRIMARY **MOOC.fi Java Programming** — free, project-based, excellent exercises

REFERENCE Oracle Java Docs (docs.oracle.com/javase/tutorial)

PRACTICE HackerRank Java domain — complete all Easy problems in Week 1

VIDEO Telusko YouTube — Java Tutorial for Beginners (selective watching)

MILESTONE

Build a basic calculator, a number guessing game, and a simple grade calculator — all from scratch, without copying.

WEEK 2 OOP Foundations & Collections

TOPICS TO COVER

- Classes and objects — constructors, instance variables, the `this` keyword
- **Encapsulation** — private fields, public getters/setters, why it matters
- **Inheritance** — `extends` keyword, method overriding, `super` keyword
- **Polymorphism** — compile-time (overloading) vs runtime (overriding)
- Abstract classes and interfaces — when to use which
- Java Collections Framework: `ArrayList`, `LinkedList`, `HashMap`, `HashSet`, `TreeMap`
- Iterating collections with for-each and iterators
- `String` class deep-dive: `StringBuilder`, common methods, immutability
- Exception handling: `try/catch/finally`, checked vs unchecked, custom exceptions

RESOURCES

- | | |
|------------------|--|
| PRIMARY | MOOC.fi — continue Part 5–9 (covers OOP thoroughly) |
| BOOK | Head First Java , 3rd Edition — Chapters 1–10 (very readable) |
| PRACTICE | LeetCode Easy: Two Sum, Reverse String, Valid Palindrome |
| REFERENCE | Baeldung.com — Java Collections Guide (well-structured articles) |

MILESTONE

Build a **Student Management System**: add/remove/search students, store in `ArrayList` / `HashMap`, handle invalid input with exceptions. No frameworks — pure Java.

02

Java Deep Dive & Modern Features

Week 3 · Write Java the way professionals write it

This week bridges the gap between beginner Java and production Java. You'll learn the features that make Java code clean, concise and industry-standard. Many tutorials skip these — don't. Interviewers and code reviews will expose gaps here.

WEEK 3 Generics, Functional Java & Modern APIs

TOPICS TO COVER

- **Generics** — generic classes & methods, bounded type parameters, wildcards
- Java 8: Lambda expressions, functional interfaces (`Predicate` , `Function` , `Consumer` , `Supplier`)
- **Stream API** — `filter` , `map` , `reduce` , `collect` , `sorted` , `distinct` , `limit`
- `Optional` — avoiding `NullPointerException` the right way
- Java Records (16+) — immutable data carriers
- Sealed classes & pattern matching basics (17+); `var` keyword (10+)
- Enhanced switch expressions
- File I/O with `java.nio.file` (`Files` , `Path` , `Paths`)
- Dates & times: `java.time` — `LocalDate` , `LocalDateTime` , `ZonedDateTime`

Why this matters for Spring Boot: Spring Boot 3+ requires Java 17. Streams are used everywhere in service layers; lambdas appear in filter chains, comparators and event handlers. Without these, Spring Boot code feels like magic and you can't debug it.

RESOURCES

- PRIMARY** **Java Brains** YouTube — Java 8 Features playlist (concise and clear)
- BOOK** **Modern Java in Action** (Urma et al.) — Chapters 3, 4, 5, 6
- PRACTICE** LeetCode — solve 5 Medium problems using Stream API instead of loops
- ARTICLE** Baeldung — Java 8 Streams, Records, Optional (individual articles)

MILESTONE

Refactor your Week 2 Student system: replace loops with Streams, add an `Optional` -based search, use Records for the model, and persist data to a JSON file with `java.nio` .

03

Spring Boot & REST API Development

Weeks 4–6 · The core of backend development

This is the phase most people are excited about — and rightly so. Spring Boot is the dominant Java framework in the industry. But don't rush it. Take three weeks, build incrementally, and understand the *why* behind every annotation before moving forward.

WEEK 4 Spring Boot Core Concepts

TOPICS TO COVER

- What Spring Framework is and why Spring Boot exists — the problem it solves
- Project setup with **Spring Initializr** (start.spring.io)
- Maven basics: `pom.xml`, dependencies, build lifecycle
- Application structure: Controller, Service, Repository layers — and why
- **Dependency Injection & Inversion of Control** — the most important concept
- `@Component`, `@Service`, `@Repository`, `@Controller`, `@RestController`
- `@Autowired` and constructor injection (prefer constructor injection)
- `application.properties` and `application.yml` — configuration files
- Spring Boot DevTools — auto-restart during development
- Logging with SLF4J and Logback

RESOURCES

PRIMARY **Spring Official Docs** — spring.io/guides (start with 'Building a RESTful Web Service')

VIDEO Amigoscode Spring Boot Tutorial (free, YouTube — 3 hour version)

BOOK **Spring Boot in Action** (Craig Walls) — Chapters 1–3

PRACTICE Spring Initializr — create 3 projects with different dependency sets

WEEK 5 Building REST APIs**TOPICS TO COVER**

- REST principles: statelessness, resource-based URLs, uniform interface
- HTTP methods: `GET`, `POST`, `PUT`, `PATCH`, `DELETE` — when to use each
- HTTP status codes: 200, 201, 204, 400, 401, 403, 404, 409, 500
- `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`
- `@PathVariable`, `@RequestParam`, `@RequestBody`, `@ResponseBody`
- `ResponseEntity` — controlling status codes and headers explicitly
- Request validation with `@Valid`, `@NotNull`, `@NotBlank`, `@Size`, `@Email`
- Global exception handling with `@ControllerAdvice` and `@ExceptionHandler`
- **DTOs** — separating API contracts from domain models
- Testing APIs with Postman or the IntelliJ HTTP Client

RESOURCES

PRIMARY **Baeldung** — REST with Spring series (baeldung.com/rest-with-spring-series)

TOOL Postman (free) — essential for testing your APIs manually

REFERENCE HTTP Status Codes — httpstatuses.com (bookmark this)

VIDEO Daily Code Buffer — Spring Boot REST API CRUD Tutorial (YouTube)

 MILESTONE

Build a fully functional **Task Management REST API**: CRUD on tasks, input validation, proper HTTP status codes, a global exception handler returning consistent error responses, data stored in-memory (no database yet).

WEEK 6 Intermediate Spring Boot**TOPICS TO COVER**

- **Spring Profiles** — dev, test, prod configurations
- `@Value` and `@ConfigurationProperties` for reading config
- Pagination and sorting with `Pageable` in Spring Data
- Basic Spring Security: adding authentication, HTTP Basic, disabling for dev
- **CORS** configuration — why it matters for frontend–backend separation
- Writing unit tests with **JUnit 5** and **Mockito**
- `@SpringBootTest` and `@WebMvcTest` for integration testing
- `MockMvc` — testing controllers without starting a full server
- **Actuator** — exposing health and info endpoints

RESOURCES**TESTING**

Baeldung — Testing in Spring Boot (comprehensive article series)

SECURITY

Spring Security official docs — Getting Started section

VIDEO

Amigoscode — Spring Boot Unit Testing with JUnit 5 and Mockito

PRACTICE

Write tests for every endpoint in your Week 5 Task API

 MILESTONE

Add profiles, CORS config and a full test suite (JUnit 5 + Mockito + MockMvc) covering every endpoint of your Task API, plus Actuator health/info endpoints.

04

Databases, SQL & JPA / Hibernate

Weeks 7–8 · Where data lives

Every backend application needs persistent storage. These two weeks cover relational databases, SQL fundamentals, and how Spring Boot talks to a database through JPA and Hibernate. Learn SQL first — don't just rely on JPA to generate queries you don't understand.

WEEK 7 SQL & Relational Databases

TOPICS TO COVER

- Relational model: tables, rows, columns, primary keys, foreign keys
- **PostgreSQL** setup — install locally, create a database, connect with `psql`
- DDL: `CREATE` / `ALTER` / `DROP TABLE`, constraints (NOT NULL, UNIQUE, CHECK)
- DML: `INSERT`, `UPDATE`, `DELETE`, `SELECT`
- Filtering: `WHERE`, AND, OR, IN, BETWEEN, LIKE, IS NULL
- Sorting & limiting: `ORDER BY`, `LIMIT`, `OFFSET`
- Aggregations: COUNT, SUM, AVG, MAX, MIN, `GROUP BY`, `HAVING`
- Joins: INNER, LEFT, RIGHT, FULL OUTER — with real examples
- Subqueries and CTEs (Common Table Expressions)
- Indexes — what they are, when to create them, performance tradeoffs
- Transactions: ACID, `BEGIN` / `COMMIT` / `ROLLBACK`, isolation levels

RESOURCES

- PRIMARY** **SQLZoo** (sqlzoo.net) — interactive exercises, complete all sections
- PRACTICE** pgexercises.com — PostgreSQL exercises with increasing difficulty
- TOOL** DBeaver (free) — visual client for writing and running SQL
- VIDEO** Hussein Nasser YouTube — PostgreSQL for Beginners

MILESTONE

Design a normalized blog schema (users, posts, comments, tags) and write **15 meaningful SQL queries** covering all join types and aggregations.

WEEK 8 Spring Data JPA & Hibernate**TOPICS TO COVER**

- **JPA vs Hibernate** — the difference between the spec and the implementation
- `@Entity` , `@Table` , `@Id` , `@GeneratedValue` , `@Column`
- Relationships: `@OneToOne` , `@OneToMany` , `@ManyToOne` , `@ManyToMany` with `@JoinColumn`
- `CascadeType` and `FetchType` — eager vs lazy loading (critical for performance)
- `JpaRepository` — `save` , `findById` , `findAll` , `deleteById`
- Derived query methods — `findByName` , `findByEmailAndActive` , etc.
- `@Query` annotation for custom JPQL and native SQL queries
- Pagination with `Page` and `Pageable`
- Database migrations with **Flyway** — versioned SQL scripts
- Spring Boot + PostgreSQL configuration in `application.yml`
- **N+1 query problem** — what it is and how to fix it with `@EntityGraph` or JOIN FETCH

RESOURCES**PRIMARY**

Baeldung — Spring Data JPA Tutorial series (comprehensive)

VIDEO

Amigoscode — Spring Data JPA (YouTube, 3 hours)

REFERENCE

Hibernate User Guide — hibernate.org/orm/documentation

TOOL

spring.io/guides — Accessing Data with JPA (official quick start)

 MILESTONE

Connect your Task API to PostgreSQL using JPA. Add Flyway migrations for schema versioning, implement pagination on `GET /tasks` , and verify N+1 isn't happening by enabling SQL logging.

05

Git, Tooling & Real-World Projects

Weeks 9–10 · Work like a professional

Technical skills alone are not enough. Employers care about how you work, not just what you build. These two weeks focus on professional workflows, tooling, and building a project that demonstrates everything you've learned.

WEEK 9 Git & Professional Workflow

TOPICS TO COVER

- Git fundamentals: `init`, `add`, `commit`, `status`, `log`, `diff`
- Branching: `branch`, `checkout`, `merge`, `rebase` — and when to use each
- Remotes: `clone`, `push`, `pull`, `fetch`, `remote`
- GitHub workflow: fork, pull requests, code reviews, merge strategies
- Branching strategies: GitFlow vs trunk-based development
- `.gitignore` — what to exclude (`target/`, `.env`, `*.class`, IDE files)
- **Conventional commits** — the commit message format used in industry
- Resolving merge conflicts — don't fear them, practice deliberately
- GitHub Actions basics — run a test suite on every push (simple CI pipeline)
- `README.md` — how to write a project README that actually gets read

RESOURCES

PRIMARY **Pro Git Book** — git-scm.com/book (free, the definitive resource)

PRACTICE learngitbranching.js.org — interactive visual Git exercises

VIDEO Fireship — Git It? How to use Git (13 min, great overview)

REFERENCE Conventional Commits — conventionalcommits.org

WEEK 10 Capstone Project — build one end to end**Option A — E-Commerce Backend API**

- **Entities:** User, Product, Category, Order, OrderItem, Cart
- **Auth:** JWT-based login and registration with Spring Security
- **Features:** product search with filters, pagination, order placement & history
- **DB / Tests:** PostgreSQL + Flyway; unit tests for services, integration tests for controllers
- **Deploy:** package as JAR, run with Docker (basic Dockerfile)

Option B — Blog Platform API

- **Entities:** User, Post, Comment, Tag, Category
- **Features:** Markdown content, post search, tag filtering, comment moderation
- **Auth:** JWT with role-based access (ADMIN manages users, USER posts)
- Email notifications on comment using Spring Mail (optional but impressive)

Option C — Job Board API

- **Entities:** Company, Job, Application, Candidate
- **Features:** job search with location/salary/type filters, application tracking
- **Auth:** two roles — Recruiter and Candidate with different permissions

Push to GitHub with a clear README, API documentation (Swagger/OpenAPI — [springdoc-openapi](#)), and at least **70% test coverage**.

AUTH

Amigoscode — Spring Boot 3 + Spring Security 6 JWT (YouTube)

SWAGGER

springdoc-openapi documentation — [springdoc.org](#)

DOCKER

Docker official Get Started guide — [docs.docker.com/get-started](#)

REFERENCE

[jwt.io](#) — understand JWT structure before implementing



Interview Prep & System Design

Weeks 11–12 · Get the job

These final two weeks translate everything you've built into interview readiness. You'll practice DSA problems, review Spring and Java fundamentals, and learn just enough system design to handle entry-level rounds.

WEEK 11 Data Structures, Algorithms & Core Concepts

DSA TOPICS (JAVA-FOCUSED)

- Arrays & Strings — two pointers, sliding window, prefix sums
- HashMaps & HashSets — frequency maps, anagram problems
- Stacks & Queues — balanced parentheses, BFS, monotonic stack
- Linked Lists — reverse, detect cycle, merge sorted lists
- Binary Search — on sorted arrays and on answer space
- Recursion & Backtracking — subsets, permutations, combination sum
- Basic Trees — traversals (in/pre/post-order), height, LCA
- Sorting — when to use which (mergesort vs quicksort vs counting sort)

JAVA & SPRING REVIEW

- JVM internals: heap vs stack, garbage collection, memory model
- Concurrency basics: threads, `synchronized`, `volatile`, `ExecutorService`
- HashMap internals — load factor, collision resolution; String pool & immutability
- Spring Bean lifecycle, scopes (Singleton vs Prototype)
- `@Transactional` — propagation types, isolation levels
- REST design — idempotency, versioning, pagination patterns

Daily schedule: 2 LeetCode problems (Easy/Medium, 45 min each) + 30 min reviewing one Java/Spring concept + 15 min writing it in your own words.

DSA

NeetCode 150 — neetcode.io (curated LeetCode list with solutions)

JAVA

JavaTpoint Interview Questions; Baeldung — Spring Interview Questions

MOCK

Pramp.com — free peer mock interviews for coding rounds

WEEK 12 System Design & Mock Interviews**SYSTEM DESIGN FUNDAMENTALS (ENTRY-LEVEL)**

- Scalability basics — vertical vs horizontal scaling
- Load balancers — what they do and common strategies (round-robin, least connections)
- Caching — where to cache, cache invalidation, Redis basics
- Databases at scale — read replicas, sharding concepts, CAP theorem
- Message queues — why they exist, Kafka/RabbitMQ concepts (no deep dive)
- API Gateway — rate limiting, authentication at the edge
- How to approach a question: requirements, estimation, components, tradeoffs

PRACTICE DESIGNS — SKETCH THESE OUT

- URL Shortener (classic entry-level question)
- A REST API for a social feed with 1M users
- A notification service

RESOURCES**BOOK****System Design Interview** (Alex Xu) — Chapters 1–5 are enough for entry-level**VIDEO**

Gaurav Sen YouTube — System Design playlist (first 8 videos)

PRACTICE

Exponent — system design practice questions with frameworks

BLOG

High Scalability — highscalability.com (real architecture case studies)

 FINAL WEEK CHECKLIST

GitHub: pin your capstone, clean README, green commit graph. **Resume:** 3–4 metric-driven bullets per project. **LinkedIn:** update headline & skills. **Apply:** 5–10 applications/day, tracked in a spreadsheet. **Mocks:** at least 3 full coding rounds on Pramp or with a peer.



Resources Master List

Everything referenced in this roadmap — bookmark this page

FREE LEARNING PLATFORMS

- **MOOC.fi** — mooc.fi/en/study/java-programming (best free Java course online)
- **Baeldung** — baeldung.com (best Spring Boot reference, bookmark everything)
- **Spring Guides** — spring.io/guides (official, hands-on, project-based)
- **SQLZoo** — sqlzoo.net (interactive SQL exercises)
- **pgExercises** — pgexercises.com (PostgreSQL-specific practice)
- **NeetCode** — neetcode.io (curated DSA practice list)
- **learngitbranching.js.org** (visual Git practice)

YOUTUBE CHANNELS

- **Amigoscode** — comprehensive Java and Spring Boot tutorials
- **Java Brains** — Java 8 features, Spring concepts, clear explanations
- **NeetCode** — algorithm walkthroughs with clean explanations
- **Gaurav Sen** — system design fundamentals
- **Telusko** — Java basics, good for quick reviews
- **Daily Code Buffer** — Spring Boot project tutorials
- **Hussein Nasser** — backend engineering, PostgreSQL, Kafka

BOOKS (FOR DEPTH, NOT PRIMARY RESOURCES)

- **Head First Java**, 3rd Edition — best introductory Java book
- **Modern Java in Action** — covers Java 8–17 features with depth
- **Spring Boot in Action** (Craig Walls) — practical Spring Boot guide
- **System Design Interview** (Alex Xu) Vol. 1 — entry-level system design
- **Pro Git** — git-scm.com/book — the only Git book you need

TOOLS TO INSTALL (ALL FREE)

- IntelliJ IDEA Community Edition · JDK 21 (LTS) · PostgreSQL 16
- DBeaver Community · Postman · Docker Desktop · Git · Maven (bundled with IntelliJ)



Projects Reference

What each project demonstrates + approximate build time

WEEK 1-2

Student Management System

3-4 hrs

Pure Java, OOP, Collections, File I/O. No framework.

WEEK 3

Enhanced Student System

2-3 hrs

Streams, Lambdas, Optionals, Records. Refactor of the previous project.

WEEK 5

Task Management REST API

6-8 hrs

Spring Boot, REST, validation, exception handling. In-memory storage.

WEEK 8

Task API + PostgreSQL

4-5 hrs

Adds JPA, Hibernate, Flyway migrations and pagination to the Week 5 project.

WEEK 10

Capstone Project

20-30 hrs

Full project: JWT auth, multi-entity domain, PostgreSQL, tests, Swagger, Docker.

💡 HOW TO USE THIS

Each project intentionally builds on the last. By Week 10 you're not starting from scratch — you're extending code you already understand. That's exactly the story you tell in interviews.



Interview Checklist

Be able to answer every one of these out loud

JAVA CORE

- ☐ Explain the difference between `==` and `.equals()`
- ☐ What is the String pool? Why are Strings immutable?
- ☐ Explain HashMap internal working — hashing, buckets, load factor, Java 8 tree bins
- ☐ Checked vs unchecked exceptions — when to use each
- ☐ Abstract class vs interface — when to use which (post Java 8)
- ☐ What is autoboxing? What are its performance implications?
- ☐ Difference between `ArrayList` and `LinkedList`
- ☐ `Collections.sort()` time complexity? What algorithm does it use?

SPRING BOOT

- ☐ What is Dependency Injection? Why does it help?
- ☐ Explain the Spring Bean lifecycle
- ☐ Difference between `@Component`, `@Service`, `@Repository` — functionally the same?
- ☐ What is `@Transactional`? What happens if you call it from within the same class?
- ☐ Explain the Spring Security filter chain
- ☐ How does JWT authentication work end to end?
- ☐ Difference between `@RestController` and `@Controller`?
- ☐ How do Spring profiles work?

DATABASES

- ☐ Explain ACID properties with an example
- ☐ What is an index? When should you add one? What are the downsides?
- ☐ Explain the N+1 query problem and how to fix it in JPA
- ☐ Difference between INNER JOIN and LEFT JOIN
- ☐ What is a transaction isolation level? Name three levels.
- ☐ What is lazy vs eager loading in JPA? Which is the default?

REST & API DESIGN

- ☐ What makes an API RESTful?
- ☐ What is idempotency? Which HTTP methods are idempotent?
- ☐ How do you version a REST API?
- ☐ What is the difference between PUT and PATCH?
- ☐ How would you handle pagination in a REST API?

GENERAL

- ☐ Walk me through your capstone — architecture, decisions, what you'd change
- ☐ How do you find and fix a slow database query in production?
- ☐ What is the difference between concurrency and parallelism?
- ☐ Explain what happens when you type a URL and press Enter (network stack)

Don't memorize answers — **explain them out loud** until they feel natural. If you built the projects in this roadmap, you already have real examples for almost every question here.

"The most important thing is consistency — one focused hour every day beats six hours on a weekend. Build things, break things, read error messages carefully, and ask good questions."

Found this useful? 

Save it, share it with someone starting their backend journey, and follow for more free roadmaps & resources.

@sovit.tech.knowledge

This roadmap covers everything you need to land your first backend job. Good luck — you've got this.